Toward an Agent Operating System: Scheduling, Memory, and Governance Primitives for Concurrent LLM Agent Deployment

Muhammad Faisal Laiq Independent Researcher
Email: shmlaiq@gmail.com

***Abstract*—Modern large language model (LLM) agents execute as unmanaged processes: they compete for context capacity without scheduling, retain no persistent state without external scaffolding, and invoke tools with permissions far exceeding task requirements. These deficiencies mirror problems that operating system research solved for conventional software decades ago. This paper proposes an Agentic Operating System (AgentOS), a unified architectural framework that maps classical OS abstractions—scheduler, memory manager, device driver, process manager, and security monitor—onto the specific demands of autonomous LLM agents. Five architectural pillars are specified: (1) an LLM Kernel implementing priority-based scheduling and context window isolation; (2) a Hierarchical Memory Subsystem extending working memory through semantic vector retrieval and append-only state logs; (3) a Tool Integration Layer standardizing peripheral access via open protocol standards; (4) a Multi-Agent Orchestration Layer defining inter-agent messaging semantics and conflict arbitration protocols; and (5) a Security and Governance Layer enforcing least-privilege tool access, PII scrubbing, and interrupt-driven human-in-the-loop checkpoints. Each pillar is formally described, positioned against concurrent systems, and evaluated for design trade-offs. The proposed framework addresses the transition from AI as a stateless inference endpoint to AI as a managed, auditable runtime—a foundational shift required before autonomous agents can be deployed reliably at enterprise scale.**

***Index Terms*—agent operating system, LLM scheduling, memory management, multi-agent orchestration, Model Context Protocol, security and governance, human-in-the-loop**

I. Introduction

Context windows run out. Agents forget. Two separate agents tasked with the same subtask burn through API budget in parallel, with no coordination layer to deduplicate the work. These are not edge cases—they are the default behavior of every LLM agent deployed today without OS-level infrastructure.

The core problem is architectural. Contemporary agent frameworks, including LangChain [1], AutoGen [2], and MetaGPT [3], treat the language model as a remote API and wrap orchestration logic in application code. This design works for single-task, short-horizon pipelines. It breaks for concurrent, stateful, multi-session deployments. A research assistant agent and a code-review agent sharing the same LLM endpoint receive no scheduling priority—the first to submit wins. Neither agent tracks what it committed to an hour ago without custom memory scaffolding.

Operating systems faced identical problems with CPU processes in the early decades of computing. Before schedulers existed, programs ran to completion, monopolizing hardware. Before virtual memory, a single process could exhaust physical RAM and crash the system. Before access control lists, any process could overwrite any file. The solutions—scheduling, virtual memory, permission enforcement—became foundational infrastructure that all software now relies on.

LLM agents need the same infrastructure layer. This paper specifies it.

The proposed Agentic Operating System (AgentOS) defines five architectural pillars, each addressing a documented failure mode in current agent deployments:

1) **LLM Kernel**: Priority-based scheduling of agent inference requests, context window isolation between concurrent agents, and a standardized system call interface for tool access.
2) **Hierarchical Memory Subsystem**: A three-tier model spanning active context (RAM equivalent), semantic vector stores (disk equivalent), and append-only state logs (commit journal equivalent).
3) **Tool Integration Layer**: Unified peripheral access via the Model Context Protocol [4], with sandboxed execution environments for code-running agents.
4) **Multi-Agent Orchestration Layer**: Structured inter-agent communication protocols and consensus mechanisms for resolving conflicting agent plans.
5) **Security and Governance Layer**: Least-privilege tool scoping, PII scrubbing, and interrupt-driven checkpoints for human-in-the-loop oversight.

The AgentOS does not replace existing agent frameworks. LangChain, AutoGen, and MetaGPT remain valid application-layer tools. The AgentOS sits below them as a kernel layer they call into.

The contributions of this paper are as follows:

- We define five formal architectural pillars for an Agentic Operating System—the first unified specification covering scheduling, memory, tool access, orchestration, and governance in a single framework.
- We map classical OS abstractions directly onto LLM agent requirements, providing principled grounding for each architectural decision.
- We identify five open research challenges—context window fairness, semantic deduplication, tool driver versioning, multi-agent consensus under factual disagreement,

and adversarial prompt injection defense—that no existing system currently resolves.

The rest of this paper is organized as follows. Section II surveys related work. Section III maps OS abstractions to agent primitives. Section IV specifies each pillar in detail. Section V analyzes design trade-offs and compares against existing systems. Section VI discusses limitations and open challenges. Section VII concludes.

II. Related Work

A. LLM Agent Frameworks

The ReAct framework [5] established the reasoning-acting loop as the standard agent execution pattern, interleaving chain-of-thought reasoning steps [6] with environment actions. On HotpotQA, ReAct reduces hallucination by grounding intermediate reasoning in retrieved Wikipedia facts, outperforming chain-of-thought-only approaches [5]. ReAct is an agent policy—it specifies how one agent should reason. It says nothing about how ten agents should share resources.

AutoGen [2] extends this to multi-agent collaboration via a conversational message-passing model, enabling complex workflows spanning code generation, mathematical reasoning, and web navigation. The gap: AutoGen provides no admission control or context window scheduling. Under heavy load, concurrent agents race for inference capacity.

MetaGPT [3] addresses role specialization by assigning agents to software engineering roles with standardized operating procedures (SOPs). On HumanEval code generation benchmarks, MetaGPT achieves 85.9% Pass@1 [3], outperforming unstructured baselines. Coordination is role-based and hardcoded; agents from different workflows cannot exchange messages without reconfiguration.

LangChain [1] is the most widely adopted agent framework, providing composable chains, tool wrappers, and memory components. It operates entirely at the application layer—there is no kernel managing resource allocation across concurrent LangChain agents.

The AIOS system [7] is the closest prior work to the AgentOS proposed here. Mei et al. [7] implement an LLM kernel providing agent scheduling, context management, and storage management. Empirical evaluation shows AIOS achieves up to 2.1× faster task execution compared to unmanaged baselines [7]. AIOS lacks a governance layer with HITL interrupt semantics and does not address PII scrubbing.

B. Memory Management for Language Models

MemGPT [8] is the foundational reference for OS-inspired LLM memory management. Packer et al. [8] map OS virtual memory concepts onto LLM context management: the bounded token window serves as fast-access working storage, while off-context external storage provides the persistent, slow-but-unlimited second tier. MemGPT implements recency-only paging and interrupt-based control flow. The AgentOS memory subsystem extends MemGPT by incorporating semantic relevance as a second retention signal, preventing eviction of highly relevant older content when recent but task-irrelevant tokens would otherwise displace it.

Retrieval-Augmented Generation (RAG) [9] provides the vector retrieval substrate for AgentOS Tier 2 memory. Lewis et al. [9] demonstrate that augmenting language models with dense vector retrieval improves factual accuracy on knowledge-intensive tasks. The AgentOS treats RAG as the warm-memory read path.

PagedAttention [10] addresses memory fragmentation at the inference engine level. Kwon et al. [10] draw a direct analogy to OS paging and apply it to KV cache management, achieving 2–4× throughput improvement over prior serving systems [10]. While PagedAttention operates below the AgentOS, its OS-inspired design philosophy validates the analogy at a complementary layer of the stack.

C. Tool Integration Standards

Toolformer [11] introduced self-supervised tool learning, training a language model to insert API calls across five tool categories: calculator, question-answering, two search engines, translation, and calendar [11]. Toolformer treats tool integration as a generation problem; the AgentOS treats it as an infrastructure problem.

The Model Context Protocol (MCP) [4] is the emerging industry standard for the tool-agent interface. Released by Anthropic in November 2024 and subsequently adopted by OpenAI, Google DeepMind, and Microsoft [4], MCP defines a JSON-RPC 2.0 transport that lets any agent connect to any MCP-compliant tool server without custom connectors. The AgentOS adopts MCP as its native tool driver interface.

D. Multi-Agent Orchestration and Evaluation

Wang et al. [12] survey the LLM agent landscape and identify multi-agent coordination as one of three core unsolved challenges alongside persistent memory and task decomposition. Neither the centralized nor decentralized orchestration paradigm specifies how to handle conflicting agent plans for the same shared resource.

AgentBench [13] tests 29 models across 8 environments including operating system interaction, database manipulation, and web browsing. Agent performance degrades sharply on tasks requiring multi-step planning beyond three to five decision points [13]. This finding directly motivates the AgentOS orchestration design, which keeps individual agent steps short and delegates coordination to a dedicated layer.

III. Background: OS Abstractions as Agent Primitives

A conventional operating system manages four physical resources via well-understood abstractions: CPU time, memory, I/O access, and inter-process communication. Table I maps each abstraction to its LLM agent equivalent.

The analogy is imperfect in one critical respect. LLM inference is stateless at the model level—the model holds no memory between calls. The OS must maintain agent state on behalf of the agent, in the same way a kernel stores process state in a process control block (PCB) when a process is not currently scheduled. The AgentOS maintains an **Agent**

TABLE I
OS-TO-AGENT ABSTRACTION MAPPING

OS Abstraction	Traditional Resource	Agent Equivalent
Scheduler	CPU cycles	Context window tokens
Memory manager	RAM / disk	Active context / vector store
Device driver	Hardware peripheral	External tool or API
IPC mechanism	Shared memory / socket	Inter-agent message bus
Access control	File permissions	Tool scope and PII boundaries
Interrupt handler	Hardware signal	HITL approval checkpoint
Process control block	CPU register state	Agent Control Record

Control Record (ACR)—a data structure persisting each agent's current task, context tier pointers, tool permissions, interrupt flags, and conversation history across inference calls.

Transformer-based language models [14] operate on sequences bounded by a fixed context window. Frontier models [15], [16] support windows of 32K to 128K tokens—finite resources that demand management.

Chain-of-thought prompting [6] reveals the core memory tension: each reasoning step consumes context window tokens. The longer the task, the more urgent the question of what to keep in context and what to page out—precisely the virtual memory problem applied to agent working memory.

IV. THE PROPOSED AGENTOS ARCHITECTURE

A. Pillar I: LLM Kernel and Scheduling

The LLM kernel treats the language model as a managed processing unit. Multiple agents submit inference requests to the kernel, which schedules them against available context window capacity and compute throughput.

Priority Queue Scheduling. Each agent inference request carries a priority tag $p \in \{\text{high}, \text{normal}, \text{low}\}$. The kernel maintains three queues and services the highest-priority non-empty queue first. Within a priority tier, requests are served FIFO. A latency-sensitive research agent receives high priority; a background audit agent receives low priority.

Round-Robin Context Allocation. For equal-priority agents, the kernel divides the available context budget C across n waiting requests:

$$c_{\text{per agent}} = \lfloor C/n \rfloor - c_{\text{ACR}} \tag{1}$$

where c_{ACR} is the fixed ACR header overhead per agent (default 512 tokens). In a 128K-token model with ten concurrent agents, each agent receives approximately 12K tokens of working context per cycle—sufficient for three to five tool calls worth of reasoning.

Admission Control. Before accepting a new agent task, the kernel checks whether total queued token demand exceeds the configured capacity threshold $T_{\max}$. Requests above this threshold are held in a backpressure buffer; the requesting process receives `BUSY` with a retry-after interval.

System Call Interface. Agents communicate with the kernel via typed system calls:

```
SYS_TOOL_CALL(agent_id, tool, params, priority
    )
SYS_MEM_READ(agent_id, query, top_k)
SYS_MEM_WRITE(agent_id, content, metadata)
SYS_AGENT_SPAWN(parent_id, task, permissions)
SYS_HITL_INTERRUPT(agent_id, action, risk)
```

This interface separates mechanism from policy. The kernel provides routing and isolation; the agent specifies intent. No agent needs to know the network address of a tool server or the current permission configuration of the system.

B. Pillar II: Hierarchical Memory Management

Two memory pathologies afflict LLM agents: **amnesia** (no recollection of prior sessions) and **confabulation** (plausible but false recollection). Three memory tiers address both.

Tier 1 — Active Context (Hot Memory). The model's context window: 8K–128K tokens depending on the deployed model. This is working memory, processed directly during inference. Promotion from Tier 2 to Tier 1 requires cosine similarity to the current task embedding τ exceeding threshold θ_{hot}.

Tier 2 — Semantic Vector Store (Warm Memory). A vector index of prior interaction summaries, retrieved documents, and completed task outcomes—the agent's persistent storage, surviving across sessions. The RAG architecture [9] provides the retrieval substrate: content blocks are embedded, indexed, and retrieved via approximate nearest-neighbor (ANN) search. Short-horizon tasks that fit entirely within Tier 1 bypass this layer; long-horizon tasks requiring prior context cannot operate without it.

Tier 3 — Append-Only State Log (Cold Memory). Every agent action—tool call issued, result received, decision made—is appended to an immutable log keyed by `(agent_id, timestamp)`. This log is the ground truth audit record. The vector store may be re-indexed or pruned; the state log is never modified. This prevents confabulation: the kernel can verify any claimed prior action against the append-only log.

Paging Policy. Content promotion from Tier 2 to Tier 1 uses a weighted score combining recency and semantic relevance:

$$\text{retain_score}(c) = \alpha \cdot \text{recency}(c) + (1-\alpha) \cdot \text{sim}(c,\ \tau) \tag{2}$$

where c is a content block, τ is the current task embedding, and $\alpha \in [0,1]$ controls the recency-relevance balance. The default value $\alpha = 0.6$ suits general-purpose agentic workloads; long-horizon planning tasks use $\alpha \approx 0.3$ (relevance dominates), while conversational tasks use $\alpha \approx 0.8$ (recency dominates). Pure recency paging — MemGPT's policy [8] — is the special case $\alpha = 1.0$.

State Commitment. Before any irreversible action, the kernel appends the agent's current reasoning chain hash to Tier 3—a write-ahead log (WAL) pattern. If the action fails, the pre-action state is fully recoverable.

C. Pillar III: Tool Integration Layer

Every tool an agent invokes—code interpreter, web browser, calendar API, database interface—is a peripheral in the AgentOS model. The Tool Integration Layer standardizes attachment and invocation via a unified interface.

MCP as the Standard Driver Interface. The AgentOS adopts the Model Context Protocol [4] as the universal tool driver interface. Each tool is exposed as an MCP server. The kernel maintains a **Tool Registry**: a runtime table of available servers, their capabilities, input schemas, and risk classifications. When an MCP server connects, it registers its tools; agents query the registry to discover what is available.

Sandboxed Execution Environments. Tools executing arbitrary code operate inside isolated containers. The kernel routes all code-execution SYS_TOOL_CALL requests through a container runtime (Docker or WebAssembly), with filesystem, network, and process namespace restricted to the minimum required for the declared task. This enforcement is unconditional.

Without this enforcement, an agent tasked with editing a Markdown file can—through a misconfigured prompt or an adversarially crafted tool result—escalate to shell access. Prompt injection attacks [17] embedded in tool outputs are a documented attack vector; the sandboxed execution environment constrains their blast radius.

Tool Driver Versioning. Each MCP server exposes a semantic version identifier. The kernel maintains a compatibility matrix, routing calls to the version the agent was configured against even when newer versions are available. This prevents silent API drift.

D. Pillar IV: Multi-Agent Orchestration

Agent Spawn and Lifecycle. A parent agent issues `SYS_AGENT_SPAWN` to create a child with a defined task and a permission set that is a strict subset of the parent's own. The child cannot acquire permissions the parent does not hold—privilege escalation through spawning is architecturally blocked. The spawn model maps to fork/join parallelism: the parent can block synchronously or poll asynchronously for the child's result.

Structured Message Bus. Agents communicate through the kernel's message bus, not shared context. Each message carries: `sender_id`, `receiver_id`, `type` $\in$ {REQUEST, RESPONSE, BROADCAST, INTERRUPT}, a JSON-schema-validated payload, and a monotonically increasing sequence number. Shared context—allowing Agent A to read Agent B's active window—is explicitly disallowed, preventing both information leakage and reasoning contamination.

Consensus and Conflict Resolution. When two agents produce conflicting outputs, the orchestration layer escalates to a **Judge Agent**. The Judge receives both outputs and their supporting reasoning chains from Tier 3 logs, and must produce a verdict $v \in \{\text{APPROVE, REJECT, DEFER}\}$ with an explicit textual rationale. The Judge's scope is constrained: it cannot spawn agents, modify permissions, or access resources beyond the two conflicting agents' outputs. This prevents a compromised Judge from taking unilateral system-wide action.

Manager Agent Pattern. For hierarchical workflows, a Manager Agent decomposes a high-level goal, spawns Workers for each subtask, monitors progress via the message bus, and merges outputs. This generalizes MetaGPT's role-based SOPs [3] to protocol-based coordination without requiring predefined role definitions.

E. Pillar V: Security and Governance

Security is embedded in the kernel's system call interface. Every agent action passes through a governance check before execution. The check is unconditional and cannot be bypassed by application code.

Least-Privilege Tool Scoping. Each agent holds a Permission Set specifying the exact tools and operations it may invoke. Any `SYS_TOOL_CALL` outside the permission set returns `PERMISSION_DENIED`. Permission sets are hierarchically scoped: a child holds at most the permissions its parent holds.

PII Scrubbing Buffer. Before any agent output is forwarded to a cloud LLM endpoint, the kernel passes it through a PII scrubbing buffer that applies named entity recognition (NER) to replace identifying entities—names, email addresses, phone numbers, account identifiers—with anonymized tokens. The original content is stored in Tier 3 under encryption; the cloud LLM receives only the scrubbed version. Token substitution is reversed when the LLM response returns.

Human-in-the-Loop Interrupts. Table II defines the Risk Taxonomy governing agent action classification:

TABLE II
AGENTOS RISK TAXONOMY AND DEFAULT POLICIES

Risk Level	Example Actions	Default Policy
Low	Read file, query DB, search	Auto-approve, log
Medium	Write file, create event	Auto-approve, alert
High	Send email, API write	HITL interrupt required
Critical	Delete data, deploy to prod	HITL + confirmation text

When an agent issues a High- or Critical-risk call, the kernel suspends the agent (preserving its ACR state), sends a structured notification to the designated approver, and blocks until approval or denial is received. Unanswered interrupts time out to denial. The approved or denied result is injected into the agent's context on resumption—the agent receives the outcome as an input, not as a silent side effect.

The HITL mechanism has a latency cost proportional to human response time. The Risk Taxonomy is the operator's control for this trade-off: teams that prioritize automation over oversight can reclassify High to Medium, accepting documented reduction in oversight.

V. ARCHITECTURE ANALYSIS

A. Lifecycle of an Agent Request

The five pillars interact through the kernel. A complete request lifecycle proceeds as:

1) User submits task. Kernel creates ACR, allocates context budget, loads Tier 2 context via `SYS_MEM_READ`.
2) Agent generates tool call via `SYS_TOOL_CALL`. Kernel validates against permission set (Pillar V).
3) Kernel checks risk classification (Pillar V, Table II). If High/Critical: HITL interrupt, agent suspends.
4) On approval, content passes PII buffer. Kernel routes call to MCP server (Pillar III) in sandboxed container if applicable.
5) Tool result returns. Kernel appends to Tier 3, injects into Tier 1 active context.
6) If agent spawns child (`SYS_AGENT_SPAWN`, Pillar IV): kernel creates child ACR with permission subset.
7) Child result returns via message bus. If child/parent conflict: Judge Agent invoked.
8) On task completion: kernel writes ACR to Tier 3, releases context budget.

B. Design Trade-offs

Isolation vs. throughput. ACR header overhead of 512 tokens per agent means ten concurrent agents consume 5K tokens in metadata. At agent counts above twenty, this overhead becomes significant. Operators reduce it by compressing permission declarations; the isolation guarantee cannot be weakened without architectural revision.

Memory freshness vs. retrieval latency. Tier 2 ANN retrieval adds latency per reasoning step that requires prior context from outside the active window. Short-horizon tasks should bypass Tier 2 entirely. Long-horizon tasks requiring weeks-old context cannot operate without it. The bypass is configurable per task class.

Oversight vs. automation speed. HITL interrupts (Pillar V) provide the strongest correctness guarantee for irreversible actions, but latency scales with human response time. The Risk Taxonomy is the primary operator control. The design makes the trade-off explicit rather than hiding it.

C. Comparison with Existing Systems

Table III compares the AgentOS against four concurrent systems.

TABLE III
AGENTOS VS. EXISTING SYSTEMS ACROSS FIVE PILLARS

System	Sched.	Memory	Tool IF	M-Agent	Gov.
AIOS [7]	✓	Partial	Custom	Limited	×
MemGPT [8]	×	Hier.	Custom	×	×
AutoGen [2]	×	Buffer	×	✓	×
MetaGPT [3]	×	×	×	Role SOP	×
LangChain [1]	×	Plugin	Wrappers	Chain	×
AgentOS	**P+RR**	**3-tier**	**MCP**	**Protocol**	**Full**

No existing system addresses all five pillars simultaneously. AIOS [7] is the closest, with demonstrated throughput gains from kernel-level scheduling, but lacks governance and PII handling. MemGPT [8] provides the most sophisticated memory model but is single-agent with no governance layer. AutoGen [2] and MetaGPT [3] address multi-agent coordination at the application layer without kernel-level resource management or security enforcement.

D. Theoretical Analysis

Although no running AgentOS implementation exists at the time of writing, the architectural choices permit formal analysis of three core properties: scheduling complexity, paging policy dominance, and context waste reduction.

Scheduling Complexity. The priority queue used by the LLM Kernel processes scheduling decisions in $O(\log n)$ per enqueue/dequeue, where n is the number of concurrently active agents. The round-robin fallback for equal-priority agents adds $O(1)$ per dispatch step. The combined per-cycle cost is $O(n \log n)$ to re-sort priorities after each completed agent turn — the same asymptotic complexity as Linux's Completely Fair Scheduler [13]. At $n = 100$ concurrent agents, this overhead is negligible relative to LLM inference latency.

Paging Policy Dominance. The retain_score function (Equation 2) Pareto-dominates pure recency paging ($\alpha = 1.0$, as in MemGPT [8]) on tasks where semantic relevance and recency are not perfectly correlated. Let $S \subset C$ be the set of content blocks that are relevant to the current task but not among the most recent. Under pure recency paging, blocks in S are evicted whenever Tier 1 is full. Under AgentOS paging with $\alpha < 1$, blocks in S are retained if their $\text{sim}(c, \tau)$ score sufficiently compensates for their lower recency score. Tasks requiring long-horizon context — legal analysis, extended code refactoring, multi-session research — retain task-critical older context that $\alpha = 1.0$ would discard.

Context Waste Reduction. In an n-agent system without ACR namespacing, each agent's intermediate reasoning can contaminate the shared context. ACR headers confine each agent's working tokens to its allocated slice $c_{\text{per_agent}} = \lfloor C/n \rfloor - c_{\text{ACR}}$. Maximum inter-agent context contamination falls from $O(C)$ (worst-case full context leak) to $O(c_{\text{ACR}} \cdot n)$ with namespacing. For a 128K-token context window shared across 10 agents with $c_{\text{ACR}} = 512$ tokens, the contamination bound drops from 128K to 5,120 tokens — a 96% reduction in maximum cross-agent token leakage.

VI. LIMITATIONS AND OPEN CHALLENGES

The AgentOS framework is a conceptual specification. No production implementation exists; the architecture has not been empirically validated on a live workload. Five specific limitations require acknowledgment.

Implementation gap. The ACR format, PII scrubbing NER pipeline, and Risk Taxonomy enforcement engine are specified at the interface level, not the implementation level. Actual performance figures for scheduling latency, Tier 2 retrieval accuracy, and HITL interrupt overhead cannot be reported without a running system. AIOS [7] provides the closest empirical reference point (2.1× throughput from kernel-level scheduling), but AIOS does not implement governance or PII scrubbing, so its numbers cannot be extrapolated directly.

Context window fairness. Round-robin allocation (Equation 1) treats all agents equally. A legal document analysis

agent requires 50K tokens of dense context; an arithmetic agent requires 200 tokens. A workload-aware scheduler analogous to Linux CFS would better serve heterogeneous populations, but designing one with formal starvation-freedom guarantees for LLM workloads is an open problem.

Semantic deduplication. The Tier 2 vector store accumulates entries from all agent sessions. Semantically equivalent content retrieved independently by multiple agents occupies redundant index space and pollutes ANN retrieval results. Online semantic deduplication at insertion time requires approximate set-membership tests over high-dimensional embeddings. No mature solution exists.

Multi-agent consensus under factual disagreement. The Judge Agent mechanism assumes conflict is resolvable given the agents' outputs and reasoning chains. This assumption fails when the conflict originates from a factual divergence: both agents retrieved different context from Tier 2 and reached internally consistent but mutually incompatible conclusions. The AgentOS specifies a three-step escalation for this case: (1) the kernel issues a targeted re-retrieval from Tier 2 using the conflicting claim from each agent as separate query vectors; (2) if re-retrieval surfaces a single context block that subsumes both conflicting claims, the Judge Agent issues an APPROVE verdict citing that block as the authoritative source; (3) if re-retrieval does not converge, the system escalates to HITL, attaching both agents' full reasoning chains and the divergent retrieved blocks to the human operator's review queue. This protocol bounds escalation cost: re-retrieval is always attempted before human involvement, and HITL is triggered only when the information needed to resolve the conflict is genuinely absent from the system's memory.

Adversarial prompt injection in tool outputs. Prompt injection attacks [17] embedded in tool results are a documented attack vector—content returned by a web browser or file reader may contain instructions designed to override the agent's system prompt. The PII buffer defends against data leakage but does not detect adversarial instructions in tool outputs. A dedicated injection detection layer operating on tool output content is a security requirement for production deployment. The AgentOS defines the required integration point — tool outputs pass through a sanitization buffer before the SYS_TOOL_CALL response is written into the agent's context — but the detection algorithm (classifier, regex, or LLM-based judgment) is operator-configured per deployment risk profile.

VII. Conclusion

LLM agents deployed without OS-level infrastructure face the same failure modes that unmanaged processes faced before operating systems existed: resource contention, state loss, privilege overreach, and no mechanism for human intervention. Five architectural pillars directly address these failures.

The LLM kernel prevents context monopolization through priority scheduling and per-agent isolation. The hierarchical memory subsystem prevents amnesia via semantic retrieval and prevents confabulation via the append-only state log. The MCP-based tool layer replaces ad hoc integrations with a standardized driver interface. The orchestration layer enables agent teams without shared-context contamination. The governance layer makes least-privilege behavior the architectural default rather than an optional enhancement.

Five open challenges — context window fairness, semantic deduplication, tool versioning, consensus under factual disagreement, and injection defense — define the research agenda for this area. Evaluation benchmarks that test agent schedulers, memory paging policies, and HITL latency trade-offs are needed alongside performance benchmarks such as AgentBench [13].

The transition from AI as a stateless inference tool to AI as a managed, auditable runtime is a prerequisite for enterprise deployment at the reliability and compliance standards that production environments require. The Agent Operating System is the infrastructure layer that makes this transition possible.

Acknowledgment

The author thanks the open-source communities behind AIOS [7], MemGPT [8], AutoGen [2], and the Model Context Protocol [4] for making their technical designs publicly available.

References

[1] H. Chase, "LangChain." GitHub, October 2022. [Accessed: Apr. 8, 2026].

[2] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," 2023.

[3] S. Hong, X. Zhuge, J. Chen, X. Cheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," 2023.

[4] Anthropic, "Model context protocol." Open Standard, Donated to Linux Foundation (AAIF), Dec. 2025, November 2024. [Accessed: Apr. 8, 2026].

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

[6] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[7] K. Mei, X. Zhu, W. Xu, W. Hua, M. Jin, Z. Li, S. Xu, R. Ye, Y. Ge, and Y. Zhang, "AIOS: LLM agent operating system," in *Proceedings of the Conference on Language Modeling (COLM)*, 2025.

[8] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," 2023.

[9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[10] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[11] T. Schick, J. Dwivedi-Yu, R. Dessí, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," 2023.

[12] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," 2023.

[13] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, H. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
[15] OpenAI, "GPT-4 technical report," 2023.
[16] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
[17] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, "Prompt injection attack against LLM-integrated applications," 2023.



Muhammad Faisal Laiq received the master's degree in telecommunication.

He has served in the government sector since 2006 and is currently Additional Director ICT at a public-sector university and hospital in Pakistan, where he leads digital transformation and ICT infrastructure. He is also an agentic AI engineer and independent researcher. His AI work centers on AI-driven development (AIDD), loop engineering, and hierarchical multi-agent systems ("agentic fleets") built with Python, FastAPI, Docker, and Kubernetes, in which a chief agent proposes specialized sub-agents but must obtain human approval before creating or activating them. His research includes the paper *Artificial Intelligence in Future Warfare: Ethical Frameworks and the Regulation of Lethal Autonomous Weapons*, and covers AI governance, forward-deployed engineering in the AI industry, biocomputing as an alternative to silicon-based AI, the environmental cost of AI resource consumption, and operating systems for cloud-accessible quantum computers.